

Тема

Препроцесори CSS (CSS preprocessors)

План

1. Загальні відомості про препроцесори CSS
2. Компіляція коду препроцесорів CSS
3. Функціональні можливості Sass (на прикладі SCSS)
4. Функції scss

Загальні відомості про препроцесори CSS

CSS препроцесори

CSS препроцесор (від англ. CSS preprocessor) - це **надбудова над CSS**, яка додає раніше недоступні можливості для CSS, за допомогою нових синтаксичних конструкцій.

Основне завдання препроцесора - це надання зручних синтаксичних конструкцій для розробника, щоб **спростити** і тим самим **прискорити розробку** та підтримку **стилів** у проектах.

CSS препроцесори перетворюють код, написаний з використанням препроцесорної мови, у чистий та валідний CSS-код.

За допомогою препроцесорів ви можете писати код, націлений на:

- читабельність;
- структурованість та логічність;
- продуктивність.

CSS препроцесори. Можливості

Існує безліч препроцесорів. Більшість з них розширює можливості чистого CSS, додаючи такі опції як:

- змінні (variables);
- домішки (mixins);
- селектори успадкування (nesting);
- арифметичні операції з параметрами;
- імпортування стилів та ін.

Ці особливості полегшують роботу з CSS: спрощують читання коду та його подальшу підтримку.

Для використання CSS препроцесора потрібно встановити CSS компілятор на ваш веб-сервер.

Декілька найпопулярніших CSS препроцесорів: [SASS](#) [LESS](#) [Stylus](#)

Загальні відомості про препроцесори CSS

Препроцесори CSS, такі як **Sass**, **LESS** або **Stylus**, є інструментами, що розширюють функціональність стандартного CSS. Вони дозволяють використовувати змінні, вкладені правила, міксіни, функції та багато інших корисних можливостей, які роблять написання стилів більш структурованим та ефективним.

Переваги використання **препроцесорів CSS**:

- **спрощують та оптимізують код CSS** (завдяки використанню змінних і міксінів можна уникнути дублювання коду та зменшити обсяг стилів)
- **підтримують масштабованість проекту** (для великих проектів дозволяють організовувати CSS у вигляді модулів, тобто розбиття на частини (partials) та імпортування їх у основний файл підвищують зручність управління стилями).
- використання змінних (для кольорів, шрифтів, відступів тощо) **полегшує підтримку дизайну** — достатньо змінити значення змінної в одному місці, щоб зміни застосувались до всього проекту;
- вкладеність (nesting) дозволяє **логічно організовувати код CSS**, подібно до структури HTML;

Загальні відомості про препроцесори CSS

- за допомогою функцій та циклів можна **автоматизувати створення стилів** (наприклад, генерація різних відтінків кольорів для темної/світлої теми);
- мають плагіни або функції для автоматичного додавання префіксів для різних браузерів (наприклад, -webkit-, -moz-), що полегшує **підтримку кросбраузерності** .
- дозволяють **продуктивніше писати код** (стилі пишуться швидше та ефективніше, що особливо важливо для великих проектів із складними дизайнами);
- **важливі для професійного старту** (багато компаній використовують препроцесори як стандарт у своїх проектах). Розуміння їх роботи може бути ключовою навичкою для працевлаштування.

Таким чином знання препроцесорів CSS робить розробника гнучкішим та ефективнішим у роботі з великими та складними проектами. Це не просто інструмент для полегшення написання стилів, а й спосіб організувати код у масштабованій і зрозумілій формі. Вивчення Sass або іншого популярного препроцесора стане чудовим доповненням до вашого технічного стеку.

Популярні препроцесори CSS

Sass (Syntactically Awesome Stylesheets) - найпопулярніший препроцесор, який підтримує два синтаксиси: класичний .sass (без фігурних дужок і крапок із комою) та сучасний .scss (подібний до стандартного CSS).

Можливості: змінні, вкладеність, міксіни, функції, цикли, імпорти.

Широко використовується у великих проектах та підтримується багатьма інструментами розробки.

LESS (Leaner Style Sheets) - препроцесор із простішим синтаксисом, який використовує JavaScript для обробки.

Можливості схожі на Sass, але з обмеженим функціоналом (наприклад, немає циклів).

Легко інтегрується в браузер (компіляція може виконуватися на стороні клієнта).

Stylus - препроцесор із гнучким синтаксисом (можна не використовувати дужки чи крапки з комою).

Дає максимум свободи в написанні стилів.

Менш популярний, але має сильні інструменти для автоматизації та функції.

Для початку рекомендується вивчати Sass, як найпопулярніший і універсальний препроцесор.

Компіляція коду препроцесорів CSS

Препроцесорна компіляція

Препроцесор CSS – це інструмент, який дозволяє писати стилі за допомогою **розширеного синтаксису** (змінні, вкладеність, міксини, функції тощо), а потім "**перекладає**" цей код у звичайний CSS, зрозумілий для браузерів. Цей процес називається **компіляцією**. Розробник пише стилі у форматі препроцесора, наприклад, **.scss** для **Sass** або **.less** для **LESS**. У цьому коді **можна використовувати додаткові можливості** препроцесора, які не підтримуються в чистому CSS.

Приклад (Sass):

```
$primary-color: #3498db;
body {
  font-family: Arial, sans-serif;
  background-color: $primary-color;

  h1 {
    color: darken($primary-color, 10%);
  }
}
```



```
body {
  font-family: Arial, sans-serif;
  background-color: #3498db;
}

body h1 {
  color: #2c7cb6;
}
```

Результат – чистий CSS:

Після компіляції отримується звичайний CSS, який можна підключити до HTML. Він не містить змінних або інших конструкцій препроцесора, лише стандартні CSS-правила.

Код препроцесора обробляється компілятором (наприклад, через термінал або інтегровані інструменти, такі як Webpack, Gulp). У результаті генерується стандартний CSS-файл.

Препроцесорна компіляція

Чому потрібна компіляція?

Браузери не розуміють мову препроцесорів, тому їхній код потрібно перекладати в звичайний CSS.

Під час компіляції препроцесор може автоматично оптимізувати CSS (наприклад, зменшувати розмір файлу шляхом видалення зайвих пробілів або коментарів).

Розробник працює з розширеним функціоналом препроцесора, але результатом є стандартний CSS, який легко інтегрується у проект.

Переваги компіляції:

Автоматичне створення оптимізованого CSS.

Легкість підтримки коду (зміни в одному місці змінюють весь згенерований CSS).

Інтеграція в сучасні інструменти для розробки (Webpack, Gulp, Vite).

Недоліки:

Потребує додаткового етапу налаштування середовища розробки (встановлення препроцесора).

Залежність від інструментів компіляції.

Популярні компілятори для SCSS/Sass

Node.js (Dart Sass). Встановлюється через npm: `npm install -g sass`

Підтримується офіційно, швидкий і простий у використанні.

Live Sass Compiler (розширення для VS Code) - автоматична компіляція у реальному часі.

Ідеально для новачків.

Webpack - використовується в сучасних проєктах через ладери (sass-loader). Підходить для складних середовищ розробки.

Gulp - інструмент для автоматизації задач із підтримкою Sass через плагін gulp-sass.

Prepros - зручний GUI-компілятор із налаштуванням без коду. Підходить для новачків, працює на Windows, MacOS, Linux.

Koala - легка програма для компіляції Sass, LESS, Stylus. Мінімалістичний інтерфейс, ідеальний для простих проєктів.

Grunt - Інструмент для автоматизації з плагіном grunt-sass. Більш складний у налаштуванні, але потужний.

Для простих задач підійдуть Live Sass Compiler, Prepros або Koala.

У великих проєктах із сучасними стеками використовуються Webpack, Gulp або Grunt.

Встановлення **Live Sass Compiler**

Live Sass Compiler — це популярне розширення для Visual Studio Code, яке дозволяє автоматично компілювати SCSS/Sass у звичайний CSS у режимі реального часу. Це зручно для швидкої розробки та перегляду результатів без потреби вручну запускати компіляцію через термінал.

Особливості Live Sass Compiler:

- автоматична компіляція .scss у .css після збереження файлу;
- генерація стисненого CSS (minified);
- можливість налаштування шляху для збереження результатів;
- підтримка повідомлень про помилки компіляції.

Щоб **встановити** Live Sass Compiler в Visual Studio Code, виконайте наступні кроки:

Відкрийте Visual Studio Code.

Відкрийте розширення. Клацніть на іконку розширень у бічній панелі (або використовуйте комбінацію клавіш Ctrl+Shift+X або Cmd+Shift+X), а потім введіть "Live Sass Compiler" у поле пошуку.

Встановіть розширення. Знайдіть "Live Sass Compiler" у результатах пошуку і встановіть його, натиснувши кнопку "Install".

Перезапустіть Visual Studio Code. Після встановлення розширення може знадобитися перезапустити VS Code, щоб зміни набрали чинності.

Налаштування Live Sass Compiler

Відкрийте налаштування VS Code: Файл → Параметри → Налаштування. Або натисніть Ctrl+,. У пошуку введіть liveSass.

Відкрийте файл **settings.json** та додайте потрібні параметри (додавши наступний блок коду:

```
"liveSassCompile.settings.formats": [  
  {  
    "format": "expanded", // формат CSS (expanded чи compressed)  
    "extensionName": ".css", // розширення файлу  
    "savePath": "/path/to/css/folder" // шлях до папки, куди зберігати CSS  
  }  
]
```

Наприклад варіант для збереження у сусідню папку css:

```
"liveSassCompile.settings.formats": [  
  {  
    "format": "expanded",  
    "extensionName": ".css",  
    "savePath": "~/../css"  
  }  
]
```

Встановлення [Live Sass Compiler](#)

Тобто потрібно [налаштувати](#) формат CSS:

"[format](#)": "expanded" - для зручного форматування,

"[format](#)": "compressed" для стиснутого формату,

[розширення](#) файлу (".css")

[шлях](#) для збереження скомпільованих файлів ("savePath").

Запустити компіляцію (натиснувши внизу кнопку "Watch Sass"). Далі відкрийте файл Sass або Scss і зберегти його. Live Sass Compiler автоматично скомпілює його в CSS у вказаний формат і збереже результат.

A blue rectangular button with a white circular icon containing a magnifying glass over a document, followed by the text "Watch Sass" in white.

Тобто ви можете почати використовувати [Live Sass Compiler](#) для автоматичної компіляції файлів Sass або Scss при збереженні.

Функціональні можливості Sass (на прикладі SCSS)

Змінні (Variables)

Змінні дозволяють зберігати значення, такі як **кольори**, **шрифти** чи **будь-які інші стилі**, для подальшого використання в стилях. Змінні оголошуються за допомогою знаку долара **\$**, після якого вказується ім'я змінної та її значення.

```
$primary-color: #3498db; // змінна для основного кольору
```

```
$font-stack: Arial, sans-serif; // змінна для шрифтів
```

```
body {  
  background-color: $primary-color;  
  font-family: $font-stack;  
}
```

У цьому прикладі ми створюємо змінні **\$primary-color** та **\$font-stack**, які використовуються в CSS-стилях для визначення фону та шрифтів. Якщо ви хочете змінити колір чи шрифт, вам достатньо змінити значення змінної, і всі стилі оновляться автоматично.

Змінні, оголошені **на верхньому** рівні SCSS-файлу **мають глобальну область видимості**, тобто доступні для використання в будь-якому місці цього файлу.

Змінні (Variables)

Змінні в Sass є **незмінними** після оголошення. Однак, їх можна **перевизчати** в межах одного файлу або всередині вкладених блоків.

```
$primary-color: #3498db;  
h1 {  
  color: $primary-color;  
}
```

```
$primary-color: #e74c3c; // перевизначення змінної
```

```
h2 {  
  color: $primary-color; // #e74c3c  
}
```

Змінні (Variables)

Змінні можуть містити **математичні операції та вирази**. Є можливість виконувати арифметичні операції з числами та використовувати результати як значення для змінних.

```
$base-size: 16px;  
$spacing: $base-size * 2; // 32px
```

```
.container {  
  padding: $spacing;  
}
```

Sass підтримує **масиви** та **карти** як типи значень для змінних, що дозволяє зберігати набори значень і працювати з ними.

Приклад з масивом:

```
$colors: red, blue, green;  
div {  
  color: nth($colors, 2); // виведе "blue"  
}
```

Приклад з картою:

```
$font-sizes: (  
  small: 12px,  
  medium: 16px,  
  large: 20px  
);  
  
div {  
  font-size: map-get($font-sizes, medium); // "16px"  
}
```

Змінні (Variables)

У SCSS також можна використовувати змінні для керування префіксами CSS-властивостей.

```
$prefix: -webkit-;
```

```
.button {  
  #{ $prefix }border-radius: 5px; // виведе "-webkit-border-radius: 5px;"  
}
```

Отже **змінні** в Sass — це **потужний інструмент** для зберігання значень, які можуть бути використані повторно в різних частинах стилів. Вони **допомагають** зберігати гнучкість і зручність у підтримці коду, а також дозволяють **легко змінювати стилі** по всьому проекту за допомогою зміни значень лише в декількох місцях.

Змінні (Variables). Інтерполяція scss

Інтерполяція в SCSS — це спосіб **динамічного вставлення значень у селектори, властивості**, або їх **значення**. Вона здійснюється за допомогою синтаксису **#{\$expression}**, де expression — це змінна, функція чи будь-який інший код SCSS, який обчислюється.

Переваги:

- дозволяє створювати більш гнучкий і динамічний код;
- полегшує роботу з повторюваними елементами, значеннями або структурами;
- забезпечує легкість підтримки та масштабування стилів.

Використовується:

У селекторах (дозволяє створювати динамічні назви селекторів).

\$name: "button";

```
#{ $name }-primary {  
  color: blue;  
}
```



Згенерований CSS:

```
.button-primary {  
  color: blue;  
}
```

Змінні (Variables). Інтерполяція scss

У властивостях (дозволяє створювати динамічні назви CSS-властивостей).

```
$prop: "margin";  
.container {  
  #{$prop}-top: 20px;  
}
```



Згенерований CSS:

```
.container {  
  margin-top: 20px;  
}
```

У значеннях властивостей (дозволяє підставляти значення змінних або виразів).

```
$size: 10px;  
.box {  
  width: #{$size} * 2; // Застосовується обчислення  
  height: #{$size}; // Вставка значення змінної  
}
```



Згенерований CSS:

```
.box {  
  width: 20px;  
  height: 10px;  
}
```

У назвах класів (дозволяє підставляти значення змінних для динамічної генерації назв класів).

```
$component-name: "button";  
.#{$component-name}-primary {  
  background-color: blue;  
  color: white;  
}
```



Згенерований CSS:

```
.button-primary {  
  background-color: blue;  
  color: white;  
}
```

Вкладеність (Nesting)

Sass дозволяє **вкладати** правила стилів у інші правила, що полегшує організацію та читаємість коду.

Вкладеність дозволяє писати код більш структуровано.

У цьому прикладі правила для `<ul>`, `<li>`, та `<a>` знаходяться всередині правила для `nav`.

Додатково, за допомогою `&:hover` ми змінюємо стиль елемента при наведенні.

Оператор `&` для посилання на батьківський селектор дозволяє звертатися до поточного батьківського селектора.



```
nav {  
  background-color: #333;  
  
  ul {  
    list-style-type: none;  
    padding: 0;  
  }  
  
  li {  
    display: inline-block;  
  }  
  
  a {  
    color: white;  
    text-decoration: none;  
    padding: 10px;  
  
    &:hover {  
      background-color: #555;  
    }  
  }  
}
```

Наслідування (Inheritance)

Sass дозволяє одному класу **успадковувати** стилі іншого класу за допомогою **@extend**.

```
.button {  
  padding: 10px 20px;  
  background-color: #3498db;  
  color: white;  
}
```

```
.primary-button {  
  @extend .button;  
  background-color: #e74c3c;  
}
```

В даному прикладі клас `.primary-button` **успадковує** стилі з класу `.button`, але **з додатковим зміненням кольором фону**. Це дозволяє уникнути дублювання коду і використовувати спільні стилі.

Міксини (Mixins)

Міксини дозволяють створювати **повторно використовувані** блоки стилів, що зберігають можливість передавати параметри.

// Міксин для тексту з градієнтною заливкою

```
@mixin gradient-text($color1, $color2, $direction: to right) {  
    background: linear-gradient($direction, $color1, $color2);  
    -webkit-background-clip: text;  
    color: transparent;  
}
```

// Використання міксина для стилізації тексту

```
h1 {  
    @include gradient-text(#3498db, #e74c3c); // градієнт з синього в червоний  
}
```

```
p {  
    @include gradient-text(#f39c12, #8e44ad, to bottom); // градієнт з жовтого в фіолетовий з  
    напрямком зверху вниз  
}
```

Міксини (Mixins). Директива @content

Директива **@content** в SCSS використовується всередині міксина для того, щоб вставляти блоки CSS-коду, передані в цей міксин під час його виклику. Іншими словами, вона дозволяє передавати вміст в міксин під час його використання.

У міксині задається місце, куди буде вставлено переданий код за допомогою **@content**. Під час виклику міксина можна додати конкретний CSS-код, який вставиться у визначене місце. Приклад:

```
@mixin example-mixin($color) {  
  background-color: $color;  
  @content;  
}  
  
.button {  
  @include example-mixin(blue) {  
    color: white;  
    border: 1px solid darkblue;  
  }  
}
```

Результат у згенерованому CSS:

```
.button {  
  background-color: blue;  
  color: white;  
  border: 1px solid darkblue;  
}
```

Функції (Functions)

Sass дозволяє створювати власні функції для виконання певних обчислень або маніпуляцій з даними.

```
@function calculate-rem($px) {  
  $base-size: 16px;  
  @return $px / $base-size * 1rem;  
}
```

```
body {  
  font-size: calculate-rem(18px);  
}
```

Тут функція `calculate-rem` дозволяє перетворювати пікселі у рем (відносні одиниці вимірювання). Це корисно для створення більш адаптивних і гнучких стилів.

Умовні оператори (Conditionals)

Sass дозволяє використовувати умови для застосування різних стилів.

```
$theme: dark;
```

```
body {  
  @if $theme == dark {  
    background-color: black;  
    color: white;  
  } @else {  
    background-color: white;  
    color: black;  
  }  
}
```

У цьому прикладі ми використовуємо умовний оператор **@if** для перевірки значення змінної **\$theme** і вибору стилів в залежності від цього значення.

Цикли (@for, @each, @while)

Цикли дозволяють генерувати стилі на основі певних умов або ітерацій.

Приклад **@for**:

```
@for $i from 1 through 3 {  
  .box-#{ $i } {  
    width: 100px * $i;  
  }  
}
```

Приклад **@each**:

```
$colors: red, green, blue;
```

```
@each $color in $colors {  
  .text-#{ $color } {  
    color: $color;  
  }  
}
```

Імпорт

У CSS спільноті, імпортування є несхвальним, тому що воно вимагає кількох HTTP запитів. Проте, імпорт, з використанням препроцесорів працює зовсім інакше. Якщо ви імпортуєте файл, використовуючи препроцесор, то він включить імпортований файл в CSS код протягом компіляції, створюючи при цьому лише один CSS файл.

Майте на увазі, що імпортування звичайного .css файлу виконується так:

```
@import 'file.css';
```

Домішки (mixin) і змінні також можна імпортувати і використовувати в основному коді. Імпортування надає можливість створювати структури файлів.

```
/* content of file1.scss */
body {
  background: #EEE;
}
/* some else SASS file */
@import "reset.css";
@import "file1.scss";

p {
  background: #0982C1;
}
```



Компільований CSS

```
@import "reset.css";
body {
  background: #EEE;
}
p {
  background: #0982C1;
}
```

Імпорт partials

У Sass імпорт файлів працює таким чином, що файли з початковим символом нижнього підкреслення **_ (підкреслення)** вважаються частинами (**partials**), тобто **допоміжними файлами**, які **не компілюються самостійно в окремі CSS-файли**.

Це особливість, яка допомагає структурувати код. Такі файли можна імпортувати до основного SCSS-файлу, і їхній вміст буде включений у кінцевий згенерований CSS.

Файли, що починаються з **_** ігноруються компілятором:

Якщо файл починається з **_** (наприклад, `_variables.scss`), компілятор Sass не створює для нього окремого CSS-файлу. Це означає, що файл призначений лише для імпорту в інші файли.

Спрощення запису:

Під час імпорту файлів з підкресленням **_** у **@import** не потрібно вказувати **_** або розширення `.scss`. Sass автоматично розуміє, що ви імпортуєте частину. Наприклад:

```
@import "variables";
```

Це те саме, що й:

```
@import "_variables.scss";
```

@use vs @import

Директиви **@import** і **@use** у Sass виконують подібні завдання — дозволяють підключати файли в проєкті, але працюють по-різному. Ось основні відмінності:

1. **@import** (застаріла директива)

Призначення: імпортує вміст одного файлу в інший, вставляючи його "як є".

Повторення коду: якщо той самий файл імпортується у кілька місць, код дублюється, що може призводити до конфліктів і зайвого роздування фінального CSS.

Глобальність: усі змінні, міксини та функції з імпортованого файлу стають доступними глобально.

Приклад використання @import:

```
// _variables.scss
```

```
$primary-color: #3498db;
```

```
// _components.scss
```

```
@import 'variables';
```

```
.button {  
  background-color: $primary-color;  
}
```

Усі змінні та міксини додаються до глобального простору імен, що може спричинити конфлікти.

@use vs @import

2. @use (рекомендована директива)

Призначення: імпортує файл, але всі змінні, міксини та функції зберігаються в окремому просторі імен (namespace), запобігаючи конфліктам.

Не дублює код: імпортований файл додається лише один раз, незалежно від того, скільки разів використовується.

Скорочення імен: можна скоротити простір імен за допомогою as.

Приклад використання @use:

```
// _variables.scss
```

```
$primary-color: #3498db;
```

```
// _components.scss
```

```
@use 'variables' as *;
```

```
.button {  
  background-color: variables.$primary-color;  
}
```

@use vs @import

Простір імен із **префіксом** (variables.\$primary-color) запобігає глобальним конфліктам.

Можна скоротити простір імен:

```
@use 'variables' as vars;
```

```
.button {  
  background-color: vars.$primary-color;  
}
```

3. Основні відмінності

Функція	@import	@use
Стан	Застаріла, не рекомендується	Сучасна, рекомендована
Глобальний доступ	Усі змінні глобальні	Змінні доступні через простір імен
Конфлікти імен	Можливі	Уникнені через простір імен
Дублювання коду	Так	Ні

Рекомендація: використовуйте @use для кращої організації коду, уникнення конфліктів і відповідності сучасним стандартам Sass.

Коментарі в SCSS

У SCSS **коментарі** використовуються **для документування коду та пояснення логіки**, що може бути корисним для розробників. SCSS підтримує два типи коментарів:

1. Однорядкові коментарі (//). Такі коментарі **не додаються** до згенерованого CSS-файлу. Використовуються для пояснення коду без впливу на фінальний файл.

// Це однорядковий коментар

\$primary-color: #3498db; // Визначення основного кольору

У згенерованому CSS ці коментарі **не будуть присутні**.

2. Багаторядкові коментарі (/* */)

Такі коментарі **додаються до згенерованого CSS-файлу**, якщо ви не вказуєте спеціальні налаштування компілятора. Використовуються для більш детальної документації, яка має бути доступною у фінальному CSS.

/* Це багаторядковий коментар

Вказується в SCSS та з'явиться у фінальному CSS */

\$secondary-color: #e74c3c;

/* Опис стилів для тіла */

body { background-color: \$secondary-color; } /* Колір фону */

У згенерованому CSS коментар **залишиться**:

/* Це багаторядковий коментар

Вказується в SCSS та з'явиться у фінальному CSS */

body { background-color: #e74c3c; } /* Колір фону */

Коментарі в SCSS

3. Коментарі з директивою !important. Компілятор Sass дозволяє зберігати деякі коментарі в CSS навіть із налаштуванням видалення звичайних коментарів. Для цього використовується директива `/*! */`

`/*! Цей коментар залишиться в CSS */`

```
body { background: #fff; }
```

У згенерованому CSS цей коментар залишиться:

`/*! Цей коментар залишиться в CSS */`

```
body { background: #fff; }
```

4. Особливості використання:

Видалення коментарів під час компіляції:

У більшості випадків однорядкові коментарі (`//`) видаляються автоматично.

Багаторядкові коментарі (`/* */`) можуть залишатися, залежно від налаштувань компілятора.

Наприклад, при компіляції з режимом `compressed` усі коментарі, крім `/*! */`, зазвичай видаляються.

Приклад налаштування компіляції для залишення коментарів:

У Live Sass Compiler можна вказати режим:

```
"outputStyle": "expanded"
```

Рекомендації щодо використання:

Однорядкові коментарі (`//`) - для тимчасових приміток і пояснень у коді, які не потрібні в згенерованому CSS.

Багаторядкові коментарі (`/* */`) - для документування важливих частин коду, які мають залишатися у фінальному CSS (наприклад, для сторонніх розробників чи замовників).

Функції SCSS

Функції SCSS

Sass підтримує широкий **набір функцій**, які допомагають зробити **роботу з кольорами, числами, рядками, списками та картою** простішою та гнучкішою.

1. Функції для роботи з кольорами

lighten(\$color, \$amount) - освітлює колір.

darken(\$color, \$amount) - затінює колір.

mix(\$color1, \$color2, \$weight) - змішує два кольори.

rgba(\$color, \$alpha) - додає прозорість до кольору.

adjust-hue(\$color, \$degrees) - змінює відтінок кольору.

desaturate(\$color, \$amount) - зменшує насиченість кольору.

Приклад:

```
$primary-color - #3498db;
```

```
.button {  
  background-color: lighten($primary-color, 10%);  
  border-color: darken($primary-color, 15%);  
}
```

Функції scss

2. Функції для роботи з числами

`percentage($number)` - перетворює число у відсоток.

`ceil($number)` - округляє до більшого.

`floor($number)` - округляє до меншого.

`abs($number)` - повертає абсолютне значення числа.

```
$base-size: 16px;
```

```
.container {
```

```
  width: percentage(0.5); // Результат - 50%
```

```
  font-size: ceil($base-size / 3); // Результат - 6px
```

```
}
```

3. Функції для роботи з рядками

`str-length($string)` - повертає довжину рядка.

`to-upper-case($string)` - перетворює рядок у верхній регістр.

`to-lower-case($string)` - перетворює рядок у нижній регістр.

```
$font-family - 'Arial';
```

```
.title {
```

```
  font-family: to-upper-case($font-family); // Результат - 'ARIAL'
```

```
}
```

Функції scss

4. Функції для списків

`length($list)` - повертає кількість елементів у списку.

`nth($list, $n)` - повертає елемент під індексом `$n`.

`join($list1, $list2)` - об'єднує два списки.

```
$colors: red, blue, green;
.text {
  color: nth($colors, 2); // Результат - blue
}
```

5. Функції для карт

`map-get($map, $key)` - повертає значення для ключа.

`map-keys($map)` - повертає всі ключі карти.

`map-values($map)` - повертає всі значення карти.

`map-has-key($map, $key)` - перевіряє, чи існує певний ключ у мапі (асоціативному масиві).

Повертає `true`, якщо ключ є, і `false`, якщо його немає

```
$theme-colors: (
  primary: #3498db,
  secondary: #2ecc71 );
.button {
  background-color - map-get($theme-colors, primary); // Результат - #3498db
}
```


Функції scss

6. Інші корисні функції

`if($condition, $true, $false)` - тернарний оператор.

`unit($number)` - повертає одиницю виміру числа.

`random($limit)` - повертає випадкове число.

```
$random-width: random(100);
```

```
.block {
```

```
    width: $random-width + px; // Результат - випадкова ширина до 100px
```

```
}
```

Sass-функції дозволяють писати динамічні стилі, зменшувати дублювання коду і підвищувати його гнучкість.

Підсумок

1. Отримали загальні поняття про препроцесори CSS
2. Навчилися налаштовувати компілятор коду препроцесорів CSS
3. Вивчили функціональні можливості Sass (на прикладі SCSS)
4. Познайомилися з основними функціями Sass

Як вам препроцесори CSS?